

HOME MONITORING SYSTEM – PROJECT REPORT

Created with: Flutter (Dart) — Mobile App & Dashboard · Arduino C / Arduino framework (ESP32/Arduino boards) — Firmware

1. Introduction

This Home Monitoring System is a combined mobile + embedded solution. The mobile app and user interface are developed in Flutter (Dart) for cross-platform support (Android/iOS). The embedded firmware that interacts with sensors and actuators is written in Arduino C (Arduino framework), running on an ESP32/ESP8266/Arduino-compatible board. The system enables real-time monitoring, alerts, and remote control of home sensors such as motion, temperature, humidity, and gas leakage.

2. Objective

- Build a low-cost, expandable home monitoring system combining a Flutter mobile frontend and Arduino firmware.
- Provide real-time sensor monitoring, push notifications, and an intuitive mobile dashboard.
- Allow easy installation and future extension (camera, door sensors, smart plugs).

3. Components Used

- ESP32 / ESP8266 development board (Arduino-compatible)
- PIR Motion Sensor (HC-SR501)
- DHT11 / DHT22 Temperature & Humidity Sensor
- MQ-2 Gas / Smoke Sensor
- Buzzer and LED indicators
- Wi-Fi network for connectivity
- Smartphone (Android / iOS) running the Flutter app
- Optional: Firebase (for push notifications), or simple REST API server

4. System Architecture

- Sensors → ESP32 (Arduino firmware) → Wi-Fi → Cloud or Local Server → Flutter Mobile App
- Notifications: Firebase Cloud Messaging (recommended) or local push mechanisms.
- Alternative: ESP32 runs a REST endpoint or posts sensor data to a lightweight backend (Firebase Realtime Database / Firestore / HTTP REST).

5. Working Principle

1. Arduino code reads sensors at regular intervals (e.g., every 2 seconds).
2. When an abnormal reading is detected (motion, gas level above threshold, high temperature), the ESP32 triggers a local alarm (buzzer) and sends data to the cloud or directly to the mobile app.
3. The Flutter app receives updates in real time (via websockets / Firebase / polling) and shows alerts. It can also display historical data and allow user to acknowledge alarms.

6. Wiring & Setup (brief)

- DHT sensor data pin → ESP32 digital pin (with pull-up if required).
- PIR sensor Vcc → 5V, GND → GND, OUT → digital input pin.
- MQ-2 analog output → ESP32 ADC pin (use voltage divider if needed).
- Buzzer → digital output pin (with transistor if needed).
- Connect ESP32 to local Wi-Fi network via credentials stored in firmware or obtained via captive portal.

7. Sample Arduino (Arduino C) Firmware Snippet

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "YOUR_SSID";
const char* password = "YOUR_PASS";
const char* serverUrl = "http://your-server.example.com/api/sensor";

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("WiFi connected");
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (isnan(t) || isnan(h)) { delay(2000); return; }

  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(serverUrl);
    http.addHeader("Content-Type", "application/json");
    String payload = "{\"temperature\": " + String(t) + ", \"humidity\": " + String(h) + "}";
    int code = http.POST(payload);
    http.end();
  }
  delay(2000);
}
```

8. Sample Flutter (Dart) Code Snippet

```
// Simple HTTP POST example in Flutter (use http package)
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<void> sendSensorData(double temp, double hum) async {
  final url = Uri.parse('http://your-server.example.com/api/sensor');
  final resp = await http.post(url,
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'temperature': temp, 'humidity': hum}),
  );
  if (resp.statusCode == 200) {
    // success handling
  }
}
```

9. Mobile App Features (Flutter)

- Real-time dashboard showing current temperature, humidity, gas level, and motion status.
- Push notifications for alarms (via Firebase Cloud Messaging).
- History graphs and logs (use Firebase / local DB).
- Settings page to set thresholds and Wi-Fi/device management.
- Acknowledge alarms and view sensor details.

10. Advantages & Conclusion

Advantages:

- Cross-platform Flutter app — single codebase for Android and iOS.
- Flexible Arduino firmware — easy to expand sensors.
- Low cost and modular.

Conclusion:

This project demonstrates integrating embedded systems (Arduino C on ESP32) with a modern cross-platform mobile app (Flutter). It is suitable for hobbyists, student projects, and prototype smart-home deployments.

11. Further Work (extensions)

- Add camera module for image verification.
- Implement secure communication (HTTPS, token auth).
- Use MQTT for efficient real-time messaging.
- Add local fallback mode (ESP32 as access point) for offline operation.